



คำอธิบายการทำงานของโค้ด HTML

อย่างละเอียด

DevPool 2026 Project · Full-Stack Web Application



สารบัญ

- ภาพรวมของโค้ด HTML
- โครงสร้างหลักของหน้าเว็บ
- CSS และการออกแบบ
- JavaScript และการทำงาน
- การเชื่อมต่อกับ Backend (server.js)
- การทำงานร่วมกันแบบ End-to-End
- สรุปและข้อควรระวัง

1. ภาพรวมของโค้ด HTML

โค้ด HTML นี้เป็น **Single Page Application** ที่แสดงโปรไฟล์ส่วนตัวและมีฟังก์ชันการโต้ตอบกับผู้ใช้ โดยทำงานร่วมกับ Backend Server (Express.js) ผ่าน REST API

ฟีเจอร์หลักของหน้าเว็บ:

ฟีเจอร์	คำอธิบาย
โปรไฟล์ส่วนตัว	แสดงชื่อ, บทบาท, รูปภาพ และข้อความแนะนำตัว

ฟีเจอร์	คำอธิบาย
การสลับภาษา	รองรับภาษาไทยและอังกฤษ (data-th, data-en)
การเปลี่ยนธีม	รองรับ Light, Dark, Auto Mode
Live Preview	พิมพ์ข้อความแล้วเห็นตัวอย่างแบบ Real-time
ระบบ Like	กดไลค์/เลิกไลค์ พร้อมแสดงจำนวน
ราคาน้ำมัน	แสดง iframe จากเว็บบางจาก

2. โครงสร้างหลักของหน้าเว็บ

2.1 DOCTYPE และ Meta Tags

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=
  <title>ทวิษ ศรีจันทร์ · DevPool 2026 · ⚡ </title>
```

ส่วน	คำอธิบาย
<!DOCTYPE html>	ประกาศให้ Browser รู้ว่าเป็น HTML5
<html lang="th">	กำหนดภาษาเริ่มต้นเป็นไทย
<meta charset="UTF-8">	รองรับภาษาไทย (UTF-8)
<meta name="viewport">	Responsive Design รองรับทุกอุปกรณ์

ส่วน	คำอธิบาย
maximum-scale=5.0	อนุญาตให้ผู้ใช้ซูมได้
user-scalable=yes	เปิดให้ปรับขนาดหน้าได้

2.2 Font Awesome Icons

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome@6.0.0-b
```

- **Font Awesome** เป็น Library ไอคอนยอดนิยม
- ใช้แสดงไอคอนต่างๆ ในหน้าเว็บ เช่น fas fa-heart , fas fa-gas-pump

2.3 CSS Variables (ธีมหลัก)

```
:root {  
  --bg: #e8f4f8;  
  --surface: #f0f8ff;  
  --text: #1a2a3a;  
  --accent: #5b8db8;  
  --header-bg: linear-gradient(135deg, #5b8db8, #7ab0d0);  
  /* ... */  
}
```

การทำงานของ CSS Variables:

```
/* กำหนดค่าเริ่มต้น */  
:root { --bg: #e8f4f8; }  
  
/* ใช้งาน */  
body { background: var(--bg); }  
  
/* เปลี่ยนธีมด้วย data-theme */  
[data-theme="dark"] {  
  --bg: #1a2a3a; /* เปลี่ยนค่าทันที */  
}
```

```
[data-theme="auto"] {  
  --bg: #d4e8d4; /* สีธรรมชาติ */  
}
```

3. CSS และการออกแบบ

3.1 การออกแบบแบบ Responsive

โค้ดใช้ **Media Queries** เพื่อปรับแต่งหน้าตาบนอุปกรณ์ต่างๆ:

```
/* สำหรับอุปกรณ์ขนาดกลาง (Tablet) */  
@media (max-width: 820px) {  
  .card { padding: 1.2rem; }  
  h1 { font-size: 1.4rem; }  
}  
  
/* สำหรับอุปกรณ์ขนาดเล็ก (Mobile) */  
@media (max-width: 576px) {  
  .header-section { flex-direction: column; }  
  .theme-option { width: 26px; height: 26px; }  
}  
  
/* สำหรับหน้าจอแนวนอน (Landscape) */  
@media (orientation: landscape) and (max-height: 500px) {  
  .profile-avatar .avatar-img { width: 50px; height: 50px; }  
}
```

3.2 Card Layout

```
.card {  
  max-width: 820px;  
  width: 100%;  
  background: var(--surface);  
  border-radius: 24px;  
  padding: 1.5rem;  
  box-shadow: 0 12px 30px rgba(70, 130, 180, 0.12);  
}
```

3.3 Animation และ Interaction

```
/* Shimmer Effect สำหรับข้อความ */
@keyframes shimmerText {
  0%, 100% { background-position: 0% 50%; }
  50% { background-position: 100% 50%; }
}

/* Bounce Effect สำหรับไอคอน */
@keyframes bounceIcon {
  0%, 100% { transform: translateY(0); }
  50% { transform: translateY(-3px); }
}

/* Blink Effect สำหรับ Cursor */
@keyframes blink {
  0%, 100% { opacity: 1; }
  50% { opacity: 0; }
}

/* Pop-in Effect สำหรับข้อความ */
@keyframes popIn {
  0% { transform: scale(0.8); opacity: 0.5; }
  100% { transform: scale(1); opacity: 1; }
}
```

4. JavaScript และการทำงาน

4.1 IIFE (Immediately Invoked Function Expression)

```
(function() {
  "use strict";
  // โค้ดทั้งหมดอยู่ภายในนี้
})();
```

เหตุผลที่ใช้ IIFE:

- ป้องกันตัวแปรรั่วไหลไปยัง Global Scope

- สร้าง Scope ส่วนตัวสำหรับตัวแปรและฟังก์ชัน
- ใช้ "use strict" เพื่อเปิดโหมด Strict Mode

4.2 DOM References

```
const card = document.getElementById('appCard');
const themeOptions = document.querySelectorAll('.theme-option');
const statusDot = document.getElementById('statusDot');
const statusLabel = document.getElementById('statusLabel');
const themeToast = document.getElementById('themeToast');
const toastIcon = document.getElementById('toastIcon');
const toastText = document.getElementById('toastText');

const liveInput = document.getElementById('liveInput');
const liveDisplay = document.getElementById('liveDisplay');
const livePreview = document.getElementById('livePreview');

const likeBtn = document.getElementById('likeBtn');
const likeCountDisplay = document.getElementById('likeCountDisplay');
const userNameDisplay = document.getElementById('userNameDisplay');

const langToggle = document.getElementById('langToggle');
const langLabel = document.getElementById('langLabel');
```

- document.getElementById() → ดึง Element ตาม ID
- document.querySelectorAll() → ดึง Element ทั้งหมดที่ตรงกับ Selector

4.3 ระบบเปลี่ยนภาษา

```
function switchLanguage(lang) {
  currentLang = lang;
  langLabel.textContent = lang === 'th' ? 'TH' : 'EN';

  // เปลี่ยนข้อความทั้งหมดที่มี data-th และ data-en
  document.querySelectorAll('[data-th][data-en]').forEach(el => {
    const text = lang === 'th' ? el.getAttribute('data-th') : el.getAttribute('data-en');
    if (text) {
      if (['SPAN', 'STRONG', 'H1', 'H3', 'H4'].includes(el.tagName) ||
        el.classList.contains('reason-text')) {
        el.textContent = text;
      }
    }
  });
}
```

```

    } else if (el.tagName === 'SMALL') {
      el.textContent = text;
    } else if (el.tagName === 'BUTTON') {
      const childSpan = el.querySelector('span[data-th][data-en]');
      if (childSpan) childSpan.textContent = text;
    } else if (!el.querySelector('small')) {
      el.textContent = text;
    }
  }
});

// เปลี่ยน Placeholder ของ Input
const inputEl = document.getElementById('liveInput');
if (inputEl) {
  inputEl.placeholder = lang === 'th' ? '💡 ..ติชมหรือดำพินพได้เลยครับ..💡' : '💡 Typ

}

showToast('🌐', lang === 'th' ? 'เปลี่ยนเป็นภาษาไทย' : 'Switch to English');
localStorage.setItem('lang', lang);
}

```

ขั้นตอนการทำงาน:

1. รับพารามิเตอร์ `lang` ('th' หรือ 'en')
2. เปลี่ยนข้อความบนปุ่มภาษา
3. วนลูปหา Element ที่มี `data-th` และ `data-en`
4. เลือกข้อความตามภาษาที่เลือก
5. เปลี่ยน `placeholder` ของ Input
6. แสดง Toast Notification
7. บันทึกภาษาลง Local Storage

4.4 ระบบเปลี่ยนธีม

```

function applyTheme(theme) {
  const actualTheme = theme === 'auto' ? 'auto' : theme;

  // กำหนด data-theme บน Card

```

```

if (theme === 'auto') {
    card.setAttribute('data-theme', 'auto');
} else if (actualTheme === 'dark') {
    card.setAttribute('data-theme', 'dark');
} else {
    card.removeAttribute('data-theme');
}

// เปลี่ยนสถานะปุ่ม
themeOptions.forEach(btn => {
    btn.classList.toggle('active', btn.dataset.theme === theme);
});

// อัปเดต Status
updateThemeStatus(theme, actualTheme);

// บันทึกใน Local Storage
localStorage.setItem('theme', theme);
currentTheme = theme;
}

```

การทำงานของ Theme:

1. ผู้ใช้คลิกปุ่มขึ้น
↓
2. applyTheme('dark')
↓
3. card.setAttribute('data-theme', 'dark')
↓
4. CSS: [data-theme="dark"] { --bg: #1a2a3a; }
↓
5. หน้าจอเปลี่ยนสีทันที
↓
6. บันทึกใน Local Storage

4.5 Live Preview (Real-time Input)

```

liveInput.addEventListener('input', function() {
    const val = this.value.trim();
    if (val === '') {
        liveDisplay.textContent = '...';
    }
});

```



```

        livePreview.classList.remove('has-text');
    } else {
        liveDisplay.textContent = val;
        livePreview.classList.add('has-text');

        // Trigger Animation
        liveDisplay.style.animation = 'none';
        void liveDisplay.offsetHeight; // Force Reflow
        liveDisplay.style.animation = 'popIn 0.3s cubic-bezier(0.34, 1.56, 0.64, 1)';
    }
});

```

ขั้นตอนการทำงาน:

1. ผู้ใช้พิมพ์ข้อความใน Input
2. Event `input` ทำงานทุกครั้งที่มีการพิมพ์
3. ข้อความแสดงใน `liveDisplay` ทันที
4. ถ้าข้อความว่าง → แสดง "..."
5. ถ้ามีข้อความ → เพิ่มคลาส `has-text` และทำ Animation

4.6 ระบบ Like

```

async function loadUserData() {
    try {
        userNameDisplay.textContent = 'กำลังโหลด...';
        likeBtn.disabled = true;
        likeBtn.style.opacity = '0.6';

        // ดึงข้อมูลจาก JSONPlaceholder
        const response = await fetch('https://jsonplaceholder.typicode.com/users/1');
        if (!response.ok) throw new Error('Failed to load user');
        const user = await response.json();

        userNameDisplay.textContent = user.name;
        isUserLoaded = true;
        likeCount = Math.floor(Math.random() * 100) + 20;
        updateLikeDisplay();

        likeBtn.disabled = false;
    }
}

```

```

        likeBtn.style.opacity = '1';
        showToast('👤', `โหลดข้อมูล ${user.name} สำเร็จ`);
    } catch (error) {
        userNameDisplay.textContent = 'ไม่พบข้อมูล';
        likeCount = Math.floor(Math.random() * 50) + 10;
        updateLikeDisplay();
        likeBtn.disabled = false;
        likeBtn.style.opacity = '1';
    }
}

```

การทำงานของปุ่ม Like:

1. หน้าโหลด → loadUserData()
 - ↓
2. fetch('https://jsonplaceholder.typicode.com/users/1')
 - ↓
3. ได้ข้อมูลผู้ใช้ → แสดงชื่อ
 - ↓
4. สร้าง Like Count สุ่ม (20-120)
 - ↓
5. ผู้ใช้คลิก Like → isLiked = true
 - ↓
6. likeCount += 1
 - ↓
7. เปลี่ยนสีปุ่มเป็นสีแดง
 - ↓
8. เปลี่ยนข้อความเป็น "เลิกไลต์"
 - ↓
9. แสดง Toast "คุณไลค์แล้ว"

4.7 Toast Notification

```

function showToast(icon, message) {
    toastIcon.textContent = icon;
    toastText.textContent = message;
    themeToast.classList.add('show');

    if (toastTimeout) clearTimeout(toastTimeout);
    toastTimeout = setTimeout(() => {
        themeToast.classList.remove('show');
    }, 3000);
}

```

```
    }, 2500);  
  }  
}
```

- รับไอคอนและข้อความ
- แสดง Toast ด้วยคลาส `show`
- ตั้งเวลา 2.5 วินาทีให้หายไปอัตโนมัติ
- ถ้ามี Toast เก่าให้เคลียร์ Timer

4.8 Local Storage

```
// บันทึกภาษา  
localStorage.setItem('lang', lang);  
  
// ดึงภาษาที่บันทึกไว้  
const savedLang = localStorage.getItem('lang') || 'th';  
  
// บันทึกธีม  
localStorage.setItem('theme', theme);  
  
// ดึงธีมที่บันทึกไว้  
let currentTheme = localStorage.getItem('theme') || 'light';
```

การใช้ Local Storage:

- ผู้ใช้ปิดหน้าเว็บแล้วเปิดใหม่ → ภาษาและธีมยังคงเดิม
- ข้อมูลไม่หายแม้จะรีเฟรชหน้า

5. การเชื่อมต่อกับ Backend (server.js)

5.1 การเชื่อมต่อกับ JSONPlaceholder (ปัจจุบัน)

```
const response = await fetch('https://jsonplaceholder.typicode.com/users/1');
```

หากต้องการเชื่อมต่อกับ server.js:

```
// เปลี่ยนเป็น
const response = await fetch('/api/users/1');
// หรือ
const response = await fetch('http://localhost:5500/api/users/1');
```

5.2 ระบบ Like (เชื่อมกับ In-Memory Database)

```
// ปัจจุบัน: ใช้ Memory ใน Browser
likeCount = Math.floor(Math.random() * 100) + 20;

// หากเชื่อมต่อกับ server.js:
async function loadUserData() {
  // ดึงข้อมูลผู้ใช้
  const userResponse = await fetch('/api/users/1');
  const user = await userResponse.json();
  userNameDisplay.textContent = user.name;

  // ดึงยอดไลค์
  const likesResponse = await fetch('/api/likes/1');
  const likesData = await likesResponse.json();
  likeCount = likesData.likes;
  updateLikeDisplay();
}

likeBtn.addEventListener('click', async function() {
  isLiked = !isLiked;
  const action = isLiked ? 'like' : 'unlike';

  // ส่ง Request ไป Server
  const response = await fetch(`/api/likes/1`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ action: action })
  });

  const data = await response.json();
  likeCount = data.likes;
  updateLikeDisplay();
});
```

5.3 การส่งข้อความ (เชื่อมกับ server.js)

```
// ฟังก์ชันส่งข้อความไป Server
async function sendMessage(message) {
  const response = await fetch('/api/messages', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      userId: '1',
      message: message
    })
  });

  const result = await response.json();
  if (result.success) {
    console.log('บันทึกข้อความสำเร็จ:', result.data);
  }
}

// ไข้เมื่อผู้ใช้กด Enter
liveInput.addEventListener('keydown', function(e) {
  if (e.key === 'Enter') {
    const message = this.value.trim();
    if (message) {
      sendMessage(message);
      this.value = '';
      liveDisplay.textContent = '...';
      livePreview.classList.remove('has-text');
    }
  }
});
```

6. การทำงานร่วมกันแบบ End-to-End

สถานการณ์ที่ 1: ผู้ใช้โหลดหน้าเว็บ

1. Browser: GET /index.html

Server: ส่งไฟล์ HTML (Static)

2. Browser: โหลด CSS และ JavaScript

Server: ส่งไฟล์ Static

3. JavaScript: fetch('https://jsonplaceholder.typicode.com/users/1')

↓ เรียก API ภายนอก

4. JSONPlaceholder: ส่งข้อมูลผู้ใช้

↓

5. JavaScript: แสดงชื่อผู้ใช้งานบนหน้าเว็บ

↓

6. JavaScript: สร้าง Like Count สุ่ม

↓

7. แสดงยอดไลค์

สถานการณ์ที่ 2: ผู้ใช้เปลี่ยนภาษา

1. User: คลิกปุ่ม "EN"

↓

2. JavaScript: switchLanguage('en')

↓

3. วนลูป Element ที่มี data-th และ data-en

↓

4. เปลี่ยนข้อความทั้งหมดเป็นภาษาอังกฤษ

↓

5. เปลี่ยน Placeholder ของ Input

↓

6. แสดง Toast "Switch to English"

↓

7. บันทึกภาษาใน Local Storage

📌 สถานการณ์ที่ 3: ผู้ใช้เปลี่ยนธีม

1. User: คลิกปุ่ม 🌙

↓

2. JavaScript: `applyTheme('dark')`

↓

3. `card.setAttribute('data-theme', 'dark')`

↓

4. CSS: `[data-theme="dark"] { --bg: #1a2a3a; }`

↓

5. หน้าจอเปลี่ยนเป็นสีเข้มทันที

↓

6. อัปเดต Status "มืด"

↓

7. แสดง Toast "เปลี่ยนเป็นโหมดมืด"

↓

8. บันทึกธีมใน Local Storage

📌 สถานการณ์ที่ 4: ผู้ใช้พิมพ์ข้อความ

1. User: พิมพ์ "Hello" ใน Input

↓

2. Event: input ถูก trigger

↓

3. JavaScript: `liveDisplay.textContent = "Hello"`

↓

4. `livePreview.classList.add('has-text')`

↓

5. Animation popIn ทำงาน

↓

6. ผู้ใช้เห็นข้อความแบบ Real-time

📌 สถานการณ์ที่ 5: ผู้ใช้กด Like

1. User: คลิกปุ่ม Like

↓

2. JavaScript: `isLiked = true`

↓

3. `likeCount += 1`

↓

4. ปุ่มเปลี่ยนสี (เพิ่มคลาส 'liked')

↓

5. ข้อความเปลี่ยนเป็น "เลิกไลต์"

↓

6. `updateLikeDisplay()` → แสดงยอดไลค์ใหม่

↓

7. แสดง Toast "คุณไลค์แล้ว"

7. สรุปและข้อควรระวัง

✅ ข้อดีของโค้ดนี้

- **Responsive Design** - รองรับทุกอุปกรณ์ (Desktop, Tablet, Mobile)

- **Multi-language** - รองรับภาษาไทยและอังกฤษ
- **Theme Switching** - Light, Dark, Auto Mode
- **Real-time Interaction** - Live Preview แบบทันที
- **Local Storage** - จำลองภาษาและธีมที่ผู้ใช้เลือก
- **Accessibility** - มี ARIA labels, tooltips
- **Clean Code** - ใช้ IIFE ป้องกัน Global Scope
- **Error Handling** - มี fallback เมื่อโหลดข้อมูลไม่สำเร็จ

⚠ ข้อควรระวัง

- ไม่เชื่อมต่อกับ **Backend** จริง - ใช้ API ภายนอกโดยตรง
- **Like Count** เป็น **Random** - ไม่ได้บันทึกใน Database
- ไม่มี **Authentication** - ทุกคนใช้ User ID เดียวกัน
- **Iframe Security** - เว็บบางจากอาจบล็อก CORS
- **Local Storage** - ข้อมูลหายเมื่อ Clear Browser Data

🚀 แนวทางการพัฒนาเพิ่มเติม

- เชื่อมต่อกับ `/api` endpoints ของ `server.js`
- ใช้ `fetch` เรียกข้อมูลจาก Backend แทน API ภายนอก
- เพิ่มการบันทึก Like Count ใน Database
- เพิ่มระบบ Authentication (Login)
- ใช้ WebSocket สำหรับ Real-time Messages
- เพิ่ม Unit Test สำหรับ JavaScript

เอกสารนี้สร้างเมื่อ: 29 มิถุนายน 2569 เวลา 21:12